PipedPeer: Zero-Configuration Distributed Task Execution via Push-Based, Decentralised Node Coordination

submitted in partial fulfillment of the requirement
for the award of the Degree of

Bachelor of Technology
in
Information Technology

by

Yateen Vaviya
Vinayak Yadav
Sahil Gupta

under the guidance of

Prof. Rupali Sawant



Department of Information Technology
Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri-West, Mumbai-400058
University of Mumbai

2024-2025

Approval Certificate

This is to certify that the Project entitled "Major Project Title" by Student 1, Student 2 and Student 3 is approved for the partial fulfillment of Major Project - I for the Final Year Project towards obtaining the Degree of Bachelor of Technology in Information Technology from University of Mumbai.

Project Guide **External Examiner**

(signature) **(signature)**

Name: **Name:**

Date: **Date:**

Head of Department **Principal**

Seal of the Institute

Declaration

I declare that the work titled **PipedPeer: Zero-Configuration Distributed Task Execution via Push-Based, Decentralised Node Coordination** is my own work carried out under the guidance of **Prof. Rupali Sawant** at Sardar Patel Institute of Technology. This work has not been submitted elsewhere for any degree.

Yateen Vaviya	**Vinayak Yadav**	**Sahil Gupta**
2023300251	**2023300264**	**2023300072**

ABSTRACT

This project presents **PipedPeer**, a distributed task execution daemon that enables a network of heterogeneous machines to collaboratively run Python scripts without central coordination or complex cluster configuration. PipedPeer operates as a background daemon on each node; users interact with it through a thin CLI client that behaves like a smart `uv run` — except the computation may execute on a remote machine entirely.

Task distribution employs a **push-based architecture**: the sender evaluates the health of all known worker nodes in real time, selects the best available worker using a **HealthScore** metric penalising CPU usage, RAM pressure, and active job count, and dispatches the task directly to that worker over **NATS** (a lightweight publish-subscribe messaging system). Environment reproducibility is achieved through **Nix content-addressed archives (.nar)**, which allow workers that have previously seen the same package set to reuse their local Nix store at zero additional cost. Sandboxing is enforced by **bubblewrap** unprivileged Linux namespaces, requiring neither root nor a container runtime on worker machines.

Node discovery combines three concurrent sources — LAN multicast (mDNS), an optional central registry, and a persisted peer cache — ensuring that task dispatch continues even when the registry is unreachable. A hard fallback guarantee ensures that every submitted task always produces a result, falling back to local execution when no remote worker is available.

Keywords: Distributed Computing, Peer-to-Peer, Push-Based Dispatch, NATS, Nix, bubblewrap, Zero-Configuration, Health Scoring, Sandboxing.

Contents

1 Introduction **8**
1.1 Problem Statement . . . 8
1.2 Objectives . . . 8
1.3 Scope . . . 9
1.4 Technologies Used . . . 10
1.5 Assumptions and Constraints . . . 10

2 Literature Survey **12**
Paper 1 . . . 12
Paper 2 . . . 12
Paper 3 . . . 12
Paper 4 . . . 12
Paper 5 . . . 13
Paper 6 . . . 13
Paper 7 . . . 13
Paper 8 . . . 13
Paper 9 . . . 13
Paper 10 . . . 13
Paper 11 . . . 14
Paper 12 . . . 14

3 Analysis **15**
3.1 System Architecture . . . 15
3.2 Data Flow Diagram . . . 16
Level 0 — Context Diagram . . . 16
Level 1 — System Decomposition . . . 17
3.3 Use Case Diagram . . . 18
3.4 Sequence Diagram . . . 19

4 Design and Methodology **21**
4.1 Proposed System . . . 21
4.1.1 Node Discovery . . . 21

4.1.2 The Health Scoring System . . . 22
4.1.3 Push-Based Dispatch over NATS . . . 22
4.1.4 Dependency Management with Nix . . . 23
4.1.5 bubblewrap Sandboxing . . . 24
4.1.6 Result Delivery . . . 24
4.2 Algorithms Used . . . 25
4.2.1 HealthScore Penalty Computation . . . 25
4.2.2 Concurrent Peer Discovery Merge . . . 25
4.2.3 Nix Store Path Cache Reuse . . . 26
4.3 Datasets / Test Workloads Used . . . 26

5 Results and Discussion 28
5.1 Implementation . . . 28
5.1.1 Repository Structure . . . 28
5.1.2 Discovery Subsystem . . . 29
5.1.3 Scheduler and Health Subsystem . . . 29
5.1.4 Worker Subsystem . . . 29
5.1.5 Monitor Subsystem . . . 30
5.1.6 Registry Server . . . 30
5.2 Results . . . 31
5.2.1 Functional Test Results . . . 31
5.2.2 Performance Benchmarks . . . 33
5.2.3 HealthScore Behaviour Validation . . . 33
5.3 Comparative Analysis . . . 33

6 Conclusion and Future Work 35
6.1 Conclusion . . . 35
6.2 Limitations of the Current System . . . 36
6.3 Future Work . . . 36

7 Research Paper 38

8 Plagiarism Report 42

List of Figures

3.1 PipedPeer High-Level System Architecture 16
3.2 Level 0 DFD — PipedPeer Context Diagram 17
3.3 Level 1 DFD — PipedPeer Internal Decomposition 18
3.4 PipedPeer Use Case Diagram 19
3.5 PipedPeer End-to-End Task Execution Sequence Diagram 20

4.1 HealthScore Computation and Worker Selection 22
4.2 Nix `.nar` Environment Packaging and Cache Reuse 24

List of Tables

1.1 Technologies Used in PipedPeer 10

4.1 bubblewrap Sandbox Properties 24

5.1 Functional Test Results 32
5.2 Performance Benchmark Results 33
5.3 Comparative Analysis of PipedPeer vs Existing Frameworks 34

8.1 Plagiarism Check Summary 42

Chapter 1

Introduction

1.1 Problem Statement

Modern software development and data processing workloads frequently require more computational power than a single developer machine can provide. The conventional solutions — renting cloud VMs or provisioning a dedicated on-premises cluster — introduce significant operational overhead: credential management, billing, cluster maintenance, and network configuration. These barriers are especially prohibitive for small teams, student projects, and research environments where the goal is to run a script quickly, not to manage infrastructure.

At the same time, idle computational capacity is abundant. Developer laptops, lab workstations, and home servers typically sit underutilised for large portions of the day. No general-purpose tool exists that allows these heterogeneous, loosely connected machines to transparently share compute resources in a zero-configuration, reliable manner.

Existing distributed computing frameworks such as Apache Spark and Ray require a trusted, pre-configured cluster with a dedicated master node. Container-based solutions impose installation requirements (Docker, Kubernetes) that violate zero-configuration principles. Remote execution tools such as `ssh` are manual, require credential setup, and offer no automatic worker selection, sandboxing, or result retrieval.

PipedPeer: Zero-Configuration Distributed Task Execution via Push-Based, Decentralised Node Coordination is designed to solve precisely this problem. It aims to make distributed script execution as simple as running a script locally — but with the computation automatically offloaded to the best available machine on the network.

1.2 Objectives

The primary objectives of this project are:

1. To design and implement a background daemon that accepts Python scripts from a thin CLI client and executes them on the best available node in a local network, transparently returning results to the user's working directory.
2. To implement real-time worker selection using a health-scoring system that accounts

for CPU utilisation, RAM pressure, and current job load on each candidate node.

3. To achieve reproducible, portable execution environments using Nix content-addressed archives (`.nar`), enabling workers to reuse previously seen package sets at zero setup cost.
4. To enforce process isolation on worker nodes using `bubblewrap` unprivileged Linux namespaces, without requiring root privileges or a container runtime.
5. To provide reliable node discovery through a combination of LAN multicast (mDNS), an optional central registry, and a persisted peer cache, ensuring the system remains functional even when the registry is unreachable.
6. To guarantee that every submitted task always produces a result by falling back to local execution when no healthy remote worker is available.

1.3 Scope

This project covers the design and implementation of the following components:

- A Go-based background daemon (`pipedpeer`) that handles peer discovery, worker health scoring, task dispatch, sandboxed execution, and result delivery.
- A thin CLI client with two operational subcommands: `pipedpeer run` (submit a task) and monitoring commands `pipedpeer jobs` and `pipedpeer ping`.
- A three-source node discovery subsystem: LAN mDNS multicast, an optional central registry server, and a disk-persisted peer cache.
- A real-time health scoring engine that assigns a `HealthScore` to each candidate worker based on CPU usage, RAM usage, and active job count.
- A NATS-based direct task transport layer for low-latency, point-to-point job delivery between sender and worker.
- A Nix-based environment packaging system that serialises script dependencies as `.nar` archives and imports them on the worker with content-addressed cache reuse.
- A `bubblewrap` sandboxing layer that isolates each task in an unprivileged Linux namespace.
- A result delivery system that zips the worker's job directory and extracts it back to the user's local folder.
- An optional central registry server for cross-subnet peer discovery.

- Live monitoring via `pipedpeer ping` (terminal dashboard) and `pipedpeer jobs` (task history).

The following are explicitly out of scope for the current implementation: internet-scale DHT discovery, NAT traversal, support for runtimes other than Python, GPU-accelerated execution, and economic incentive mechanisms.

1.4 Technologies Used

Table 1.1: Technologies Used in PipedPeer

Technology	**Category**	**Purpose**
Go (Golang)	Core Language	Daemon, CLI, scheduler, worker, monitor — single static binary
rowgray NATS	Messaging	Direct node-to-node task delivery; lightweight pub-sub transport
Nix / .nar	Env. Packaging	Content-addressed dependency archives; zero-cost cache reuse on workers
rowgray bubblewrap	Sandboxing	Unprivileged Linux namespace isolation; no root or container daemon required
mDNS	Discovery	Zero-configuration LAN peer discovery
rowgray Cobra (Go)	CLI Framework	Subcommand structure for `run`, `jobs`, `ping`
Custom Registry	Discovery	Optional central peer registry for cross-subnet coordination

1.5 Assumptions and Constraints

The following assumptions and constraints apply to the current implementation:

- All participating nodes run the PipedPeer daemon binary. No agent or pre-configuration is required beyond running the daemon.
- The current implementation supports Python scripts. Other runtimes can be added by extending the environment packaging and invocation logic.
- Worker nodes run Linux (required for bubblewrap and Nix). The sender node may run any OS supported by Go.
- The registry server is entirely optional. The system functions without it via LAN discovery and cached peer lists.

- Nix must be installed on worker nodes for environment packaging. Workers without Nix can still receive tasks if `--isolate=false` is passed and dependencies are pre-installed.
- The sender node must remain reachable for the duration of a task for result delivery. Sender fault tolerance is deferred to future work.
- No incentive or payment mechanism is present; nodes contribute compute voluntarily.

Chapter 2

Literature Survey

This chapter surveys foundational and recent literature across the key technical domains that PipedPeer builds upon: distributed systems, peer-to-peer networking, content-addressed storage, distributed task scheduling, sandboxing and isolation, and network transport protocols. A minimum of twelve papers are reviewed.

Paper 1: *[Paper Title — Placeholder]*

Authors: [Author(s)] **Published In:** [Journal / Conference Name], [Year]

Summary: [Provide a 3–5 sentence summary of the paper's main contribution, methodology, and findings.]

Relevance to PipedPeer: [Explain specifically how this paper's contribution relates to PipedPeer — e.g., its scheduling algorithm informs the HealthScore dispatch model, or its analysis of LAN multicast protocols informed the mDNS discovery design.]

Paper 2: *[Paper Title — Placeholder]*

Authors: [Author(s)] **Published In:** [Journal / Conference Name], [Year]

Summary: [Provide a 3–5 sentence summary.]

Relevance to PipedPeer: [Explain the specific relevance.]

Paper 3: *[Paper Title — Placeholder]*

Authors: [Author(s)] **Published In:** [Journal / Conference Name], [Year]

Summary: [Provide a 3–5 sentence summary.]

Relevance to PipedPeer: [Explain the specific relevance.]

Paper 4: *[Paper Title — Placeholder]*

Authors: [Author(s)] **Published In:** [Journal / Conference Name], [Year]

Summary: [Provide a 3–5 sentence summary.]

Relevance to PipedPeer: [Explain the specific relevance.]

Paper 5: *[Paper Title — Placeholder]*

Authors: [Author(s)] **Published In:** [Journal / Conference Name], [Year]

Summary: [Provide a 3–5 sentence summary.]

Relevance to PipedPeer: [Explain the specific relevance.]

Paper 6: *[Paper Title — Placeholder]*

Authors: [Author(s)] **Published In:** [Journal / Conference Name], [Year]

Summary: [Provide a 3–5 sentence summary.]

Relevance to PipedPeer: [Explain the specific relevance.]

Paper 7: *[Paper Title — Placeholder]*

Authors: [Author(s)] **Published In:** [Journal / Conference Name], [Year]

Summary: [Provide a 3–5 sentence summary.]

Relevance to PipedPeer: [Explain the specific relevance.]

Paper 8: *[Paper Title — Placeholder]*

Authors: [Author(s)] **Published In:** [Journal / Conference Name], [Year]

Summary: [Provide a 3–5 sentence summary.]

Relevance to PipedPeer: [Explain the specific relevance.]

Paper 9: *[Paper Title — Placeholder]*

Authors: [Author(s)] **Published In:** [Journal / Conference Name], [Year]

Summary: [Provide a 3–5 sentence summary.]

Relevance to PipedPeer: [Explain the specific relevance.]

Paper 10: *[Paper Title — Placeholder]*

Authors: [Author(s)] **Published In:** [Journal / Conference Name], [Year]

Summary: [Provide a 3–5 sentence summary.]

Relevance to PipedPeer: [Explain the specific relevance.]

Paper 11: *[Paper Title — Placeholder]*

Authors: [Author(s)] **Published In:** [Journal / Conference Name], [Year]

Summary: [Provide a 3–5 sentence summary.]

Relevance to PipedPeer: [Explain the specific relevance.]

Paper 12: *[Paper Title — Placeholder]*

Authors: [Author(s)] **Published In:** [Journal / Conference Name], [Year]

Summary: [Provide a 3–5 sentence summary.]

Relevance to PipedPeer: [Explain the specific relevance.]

Chapter 3

Analysis

3.1 System Architecture

PipedPeer follows a **Daemon, Not CLI** architecture. All networking, peer discovery, health scoring, dispatch, sandboxing, and result delivery are handled by a Go daemon process running in the background on each node. The `pipedpeer` CLI is a thin client that communicates with this daemon; it carries no peer-to-peer logic of its own.

Every node simultaneously acts as both a potential **sender** and a potential **worker**. There is no dedicated master node. The sender role is activated when a user submits a task via `pipedpeer run` — the local daemon becomes the temporary manager for that task, selecting a worker, dispatching the job, and retrieving the result. The worker role is always active — the daemon continuously listens over NATS for incoming tasks.

The system is composed of five major subsystems:

1. **Discovery Subsystem:** Concurrently queries LAN mDNS, the optional registry, and the disk-cached peer list to build a candidate worker list.
2. **Scheduler / Health Subsystem:** Queries each candidate worker for real-time resource metrics and computes a `HealthScore`; selects the best worker and pushes the task via NATS.
3. **Worker Subsystem:** Accepts incoming tasks over NATS, sets up the Nix environment, creates a bubblewrap sandbox, executes the script, and delivers the result.
4. **Monitor Subsystem:** Maintains a task history store for `pipedpeer jobs` and provides the live terminal dashboard for `pipedpeer ping`.
5. **Registry Server (Optional):** A lightweight, stateless coordination server that holds only peer addresses; carries no task state, queues, or results.

[INSERT FIGURE: High-Level Architecture Diagram. Show: User's Machine running the CLI client talking to the local PipedPeer Daemon; the Daemon's five internal subsystems (Discovery, Scheduler/Health, NATS Client, Worker, Monitor); arrows from Daemon to Worker Node over NATS; optional Registry on the side with a dashed arrow from Discovery.]

Figure 3.1: PipedPeer High-Level System Architecture

3.2 Data Flow Diagram

Level 0 — Context Diagram

At the highest level of abstraction, PipedPeer has three external entities:

- **User:** Submits a Python script and working directory via the CLI; receives results in the same local folder.
- **Worker Node:** Receives tasks from the sender daemon over NATS, executes them inside a sandbox, and returns results.
- **Registry Server:** An optional coordination aid that nodes can register with and query to discover peers on other subnets.

The central process is the **PipedPeer Daemon**, which mediates all data flows between these entities.

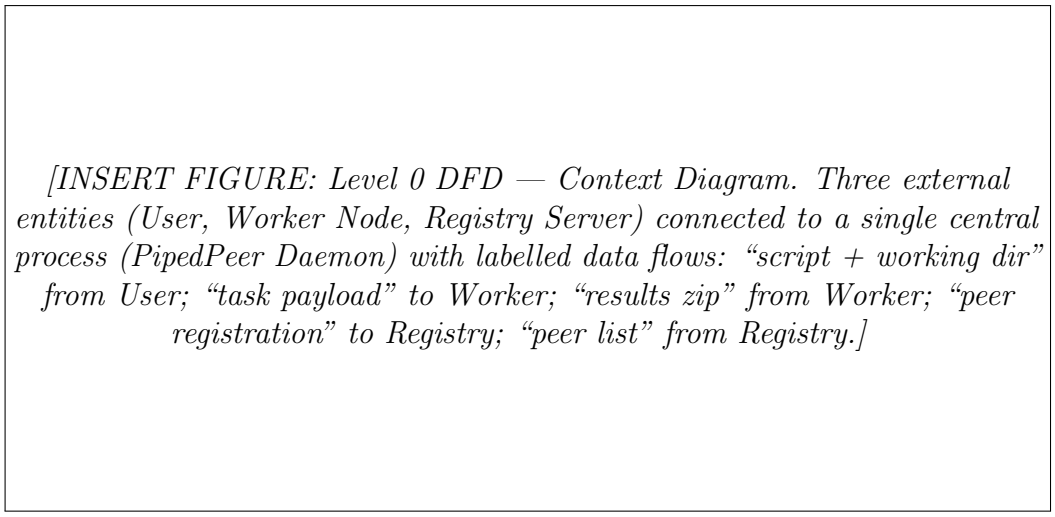


Figure 3.2: Level 0 DFD — PipedPeer Context Diagram

Level 1 — System Decomposition

Decomposing the central daemon process reveals the following internal sub-processes and data stores:

- **Process 1.0 — Peer Discovery:** Concurrently queries LAN mDNS, the registry, manual peer config, and the cached peer list; merges results into a single candidate list.
- **Process 2.0 — Health Scoring:** Queries each candidate worker's CPU usage, RAM usage, and active job count; computes a `HealthScore` (float64, starting at 1.0 with penalties).
- **Process 3.0 — Task Dispatch:** Selects the worker with the highest `HealthScore`; packages the script, working directory, and `.nar` archive into a task payload; pushes via NATS.
- **Process 4.0 — Environment Setup (Worker):** Imports the `.nar` archive into the worker's Nix store; reuses already-cached store paths by hash.
- **Process 5.0 — Sandbox & Execution (Worker):** Creates a bubblewrap unprivileged namespace; bind-mounts the working directory (read-write) and Nix environment (read-only); executes the script.
- **Process 6.0 — Result Delivery:** Worker zips the job directory; signals the sender; sender downloads and extracts the zip into the original local folder.
- **Process 7.0 — Monitoring:** Records task lifecycle events to a history store; exposes `pipedpeer jobs` and `pipedpeer ping`.

[INSERT FIGURE: Level 1 DFD — showing the 7 sub-processes above, with data stores (peer cache on disk, task history store) and data flows between processes and external entities. Show the NATS channel between Process 3.0 and Process 4.0/5.0.]

Figure 3.3: Level 1 DFD — PipedPeer Internal Decomposition

3.3 Use Case Diagram

The primary actors in PipedPeer are: the **User** (who submits tasks via the CLI), the **Worker Node** (which executes tasks), and the **Registry Server** (which assists peer discovery).

The key use cases are: Submit Task (`pipedpeer run`); View Task History (`pipedpeer jobs`); View Live Node Dashboard (`pipedpeer ping`); Discover Peers via mDNS; Register with Registry; Fetch Peer List from Registry; Query Worker Health; Dispatch Task via NATS; Import `.nar` Environment; Execute Script in bubblewrap Sandbox; Return Result Zip; and Fall Back to Local Execution.

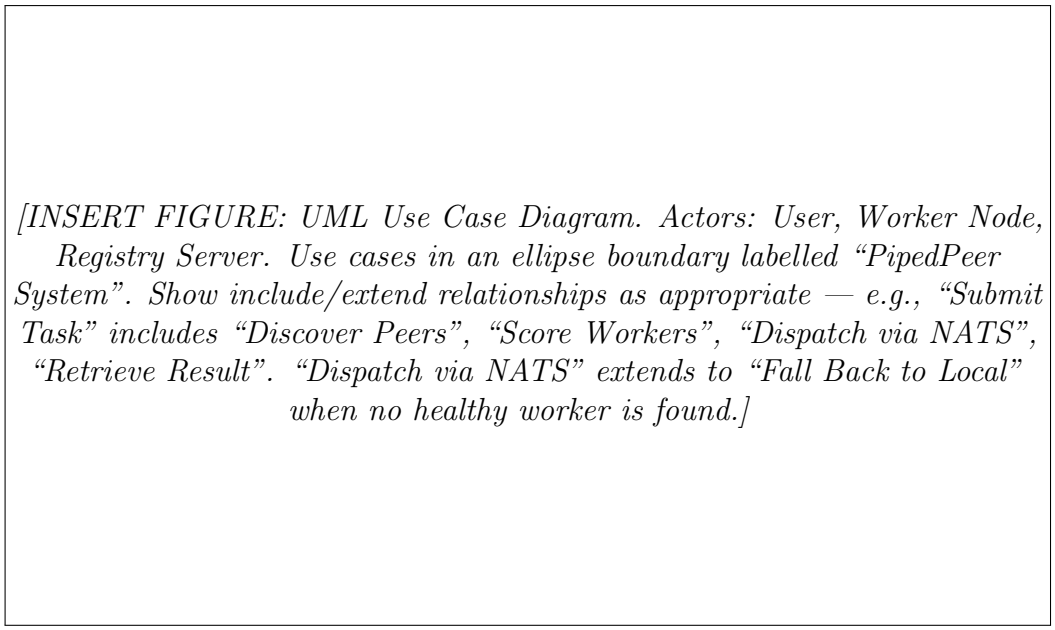


Figure 3.4: PipedPeer Use Case Diagram

3.4 Sequence Diagram

The following sequence diagram illustrates the end-to-end flow for a typical task submission and execution cycle in PipedPeer. Participants are: User (CLI), Local Daemon (Sender), Worker Node, and Registry Server (optional, shown with dashed lifeline).

The sequence of interactions proceeds as follows:

1. User runs `pipedpeer run script.py` — CLI forwards the request to the local daemon.
2. Local Daemon initiates concurrent peer discovery: broadcasts mDNS query on LAN; queries Registry (if configured); loads cached peer list.
3. Discovery results are merged into a candidate worker list.
4. Local Daemon queries each candidate worker for real-time health metrics (CPU, RAM, active jobs).
5. Local Daemon computes a `HealthScore` for each candidate; selects the worker with the highest score.
6. If no healthy remote worker is found, the daemon falls back to local execution (skips steps 7–11; executes locally).
7. Local Daemon resolves script dependencies into a Nix environment and serialises to a `.nar` archive.
8. Local Daemon pushes the task payload (script + working directory + `.nar`) to the

selected Worker Node over NATS.

9. Worker Node receives the task; imports the `.nar` archive (reusing already-cached store paths by hash).
10. Worker Node creates a bubblewrap namespace; bind-mounts the working directory and Nix environment; executes the script.
11. On script completion, Worker Node zips the job directory and signals the sender.
12. Local Daemon downloads the result zip and extracts it into the user's original local folder.
13. CLI notifies the user that the task is complete; output files are present in the working directory.

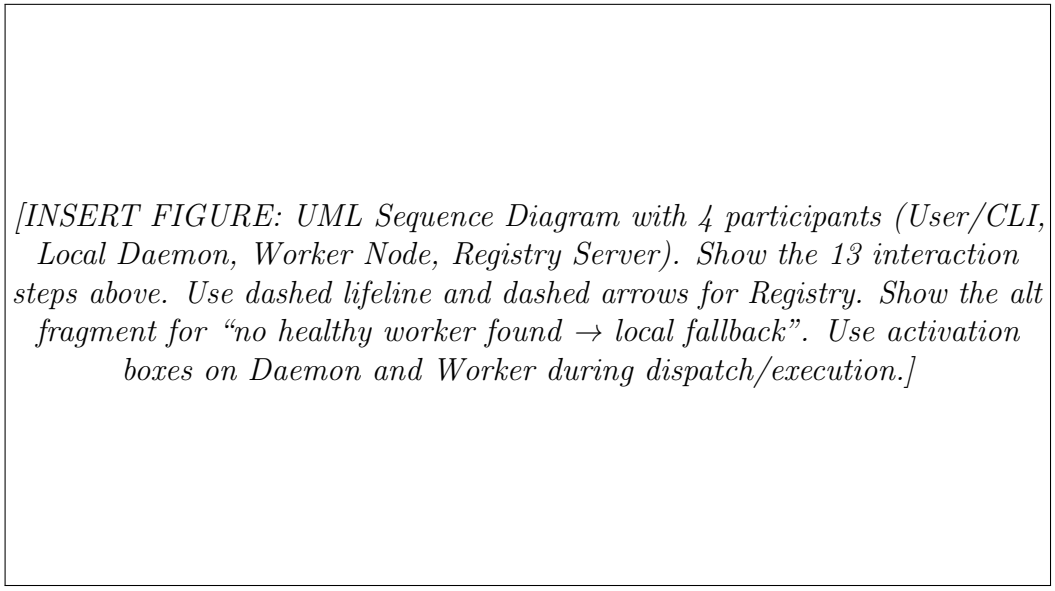


Figure 3.5: PipedPeer End-to-End Task Execution Sequence Diagram

Chapter 4

Design and Methodology

4.1 Proposed System

PipedPeer is a distributed task execution daemon built around five core design axioms: (1) the sender evaluates available workers and *pushes* the task to the best fit — workers do not bid for work; (2) Nix content-addressed caching means a worker that already has the required packages pays nothing to set up the environment; (3) every submitted task always produces a result — if no remote worker is available, the task falls back to local execution; (4) every task runs inside a bubblewrap namespace sandbox unless the user explicitly opts out; and (5) PipedPeer runs as a background daemon — the CLI is simply a thin client.

The system is decomposed into six primary subsystems, each with a clearly defined responsibility boundary.

4.1.1 Node Discovery

When the daemon starts or when a task is about to be dispatched, it simultaneously queries three discovery sources and merges the results into a single candidate list:

- **LAN mDNS Multicast:** Finds nodes on the same subnet with no configuration. Always available.
- **Registry:** Nodes self-register with a central registry server. The registry returns a list of known peers and their last-seen addresses. Available when the registry is reachable.
- **Manual Peers + Cached Peer List:** Statically configured peer addresses in the daemon config are always attempted. The last known peer list, persisted to disk, is used as a fallback when the registry is unreachable.

The daemon never blocks waiting for any single discovery source. All three are queried concurrently and their results are merged. This design ensures that the loss of the registry does not interrupt task dispatch.

4.1.2 The Health Scoring System

Each candidate worker is assigned a `HealthScore` (a `float64`) by the sender at dispatch time. The score starts at 1.0 and is reduced by penalties derived from real-time metrics queried from the worker:

$$\text{HealthScore} = 1.0 - P_{\text{cpu}} - P_{\text{ram}} - P_{\text{jobs}} \tag{4.1}$$

where:

- P_{cpu} is a penalty scaled by the worker's current CPU utilisation.
- P_{ram} is a penalty scaled by the worker's current RAM usage.
- P_{jobs} is a penalty proportional to the number of jobs currently running on the worker.

The sender computes a `HealthScore` for every candidate in the merged peer list, then dispatches the task to the node with the highest score. Nodes that are unreachable during health querying are excluded from consideration entirely.

If all reachable workers fall below an acceptable health threshold, or if no remote workers are reachable at all, the sender dispatches the task to the local machine. This is a hard guarantee: every submitted task will eventually run.

[INSERT FIGURE: HealthScore computation diagram. Show three input metrics (CPU %, RAM %, Active Jobs) flowing into a penalty computation block, producing a final HealthScore for each candidate node. Show a comparison step selecting the node with the highest score.]

Figure 4.1: HealthScore Computation and Worker Selection

4.1.3 Push-Based Dispatch over NATS

PipedPeer uses a push-based architecture. Once the best worker has been selected, the sender opens a NATS connection directly to that worker and pushes the task payload. NATS is a lightweight publish-subscribe messaging system; PipedPeer uses it for direct node-to-node communication with no broker or relay in the hot path.

The task payload contains:

- The Python script to execute.
- The working directory contents required by the script.
- The resolved Nix environment, serialised as a `.nar` archive.
- Resource requirements and metadata.

This push model places the scheduling responsibility on the sender, which has direct health visibility into every known worker. This eliminates the need for a shared job queue and reduces dispatch latency to a single round-trip.

4.1.4 Dependency Management with Nix

A worker node may never have seen the dependencies required by an incoming task. Installing packages at execution time is slow and pollutes the worker's system. PipedPeer addresses this using Nix content-addressed archives.

Nix represents every package as a content-addressed store path. A Nix Archive (`.nar`) is a serialised snapshot of one or more store paths. The flow is:

1. **Sender side:** The sender resolves the script's dependencies into a Nix environment and serialises it to a `.nar` file.
2. **Transport:** The `.nar` file is included in the task payload sent to the worker over NATS.
3. **Worker side:** The worker imports the `.nar`. Nix checks each store path hash against its local store. Paths that already exist are reused instantly — no download, no unpack.
4. **Execution:** The script is executed with the imported environment. The Nix store path is mounted read-only inside the bubblewrap sandbox.

This means the first time a worker encounters a particular environment it pays the full import cost. Every subsequent task using the same packages is effectively free to set up.

[INSERT FIGURE: Nix .nar flow diagram. Show Sender resolving dependencies → serialising to .nar → sending in payload → Worker importing .nar → Nix store hash check (cache hit / cache miss paths) → script execution with read-only store mount.]

Figure 4.2: Nix `.nar` Environment Packaging and Cache Reuse

4.1.5 bubblewrap Sandboxing

Every task runs inside a bubblewrap namespace sandbox by default. bubblewrap creates an unprivileged Linux user namespace providing filesystem, process, and network isolation without requiring root or a container daemon.

Table 4.1: bubblewrap Sandbox Properties

gray!20 **Property**	**Behaviour**
Enabled by default	Yes. All tasks are sandboxed unless `--isolate=false` is passed.
rowgray Privilege required	None. bubblewrap uses unprivileged user namespaces.
Filesystem access	Only the job working directory (read-write) and Nix environment (read-only) are visible inside the sandbox.
rowgray Network access	Inherits host network by default. Can be restricted per task.
Process isolation	The script and its children are the only processes visible inside the sandbox.

4.1.6 Result Delivery

When the task script finishes executing, the worker:

1. Zips the entire job working directory (all files present at completion).
2. Signals the sender that the task is complete.
3. The sender downloads the zip from the worker.

4. The sender extracts the zip, overwriting/adding files in the original local folder.

The user's local folder is updated in-place. There is no separate results directory — output files simply appear alongside the script that produced them, as if the script had run locally. The zip includes everything in the job directory at completion. It is the script's responsibility to write meaningful outputs to the working directory.

4.2 Algorithms Used

4.2.1 HealthScore Penalty Computation

The HealthScore assigned to each candidate worker is computed by the sender using real-time metrics queried from the worker's daemon. The algorithm proceeds as follows:

1. Query the candidate worker for its current CPU utilisation ($u_{\text{cpu}} \in [0, 1]$), RAM utilisation ($u_{\text{ram}} \in [0, 1]$), and active job count ($n_{\text{jobs}} \geq 0$).
2. Compute the penalty for each factor:

$$P_{\text{cpu}} = \alpha_{\text{cpu}} \cdot u_{\text{cpu}} \tag{4.2}$$

$$P_{\text{ram}} = \alpha_{\text{ram}} \cdot u_{\text{ram}} \tag{4.3}$$

$$P_{\text{jobs}} = \alpha_{\text{jobs}} \cdot n_{\text{jobs}} \tag{4.4}$$

   where α_{cpu}, α_{ram}, α_{jobs} are configurable penalty weights.
3. Compute the final score: $\text{HealthScore} = \max(0,\ 1.0 - P_{\text{cpu}} - P_{\text{ram}} - P_{\text{jobs}})$.
4. Workers that do not respond to the health query within a configurable timeout are assigned HealthScore = 0 and excluded from dispatch.

The sender selects the candidate with the highest `HealthScore`. If all scores fall below a minimum threshold, the task is dispatched to the local machine.

4.2.2 Concurrent Peer Discovery Merge

The three discovery sources are queried concurrently using goroutines. The merge algorithm:

1. Launch goroutines for LAN mDNS, registry query, and cached peer list load simultaneously.
2. Collect results into a shared candidate map keyed by peer address (deduplicating entries that appear in multiple sources).

3. Apply a short timeout; sources that do not respond within the timeout contribute no candidates (do not block the others).
4. Return the merged, deduplicated candidate list to the health scorer.

This algorithm guarantees that a slow or unreachable registry never delays the dispatch of a task, since LAN mDNS and the cached peer list can satisfy discovery independently.

4.2.3 Nix Store Path Cache Reuse

When the worker receives a `.nar` archive, the Nix import process performs content-addressed deduplication:

1. For each store path in the `.nar`, compute its expected Nix store hash.
2. Check the worker's local Nix store for a path with the same hash.
3. If a matching path exists (cache hit): skip unpacking; reference the existing path directly.
4. If no matching path exists (cache miss): unpack the store path from the `.nar` into the Nix store.
5. After all paths are resolved, mount the environment read-only inside the bubblewrap namespace.

The time complexity of setup is $O(m)$ for a cache miss on m new packages, and $O(1)$ for a full cache hit. On repeat invocations with identical dependency sets, setup cost approaches zero.

4.3 Datasets / Test Workloads Used

PipedPeer is a general-purpose task execution layer and does not operate on fixed datasets. Inputs are arbitrary Python scripts and their working directories. For testing and benchmarking during development, the following workloads are used:

- **`hello.py`:** A minimal smoke-test script that writes a text file to the working directory. Used to verify end-to-end task dispatch, sandbox isolation, and result delivery.
- **`two_node_local.py`:** A script that launches two PipedPeer daemon instances on the same machine and submits a cross-dispatch task between them. Used to verify NATS communication, health scoring, and peer discovery on a single host.
- **[INSERT ADDITIONAL TEST WORKLOAD]:** [Describe any additional scripts used for performance benchmarking — e.g., a CPU-intensive computation used to

measure task execution time across different worker counts, or a dependency-heavy script used to benchmark `.nar` import times.]

Chapter 5

Results and Discussion

5.1 Implementation

The PipedPeer implementation was completed across the following major components.

5.1.1 Repository Structure

The codebase is organised into two top-level modules: `daemon/` (the Go daemon and CLI) and `registry/` (the optional registry server). The daemon module is further divided into `config/`, `discovery/`, `scheduler/`, `worker/`, `monitor/`, and `nats/` packages, each with a single clearly defined responsibility.

Listing 5.1: PipedPeer Repository Structure

```
pipedpeer/
|
|-- daemon/
|   |-- main.go
|   |-- config/
|   |   `-- config.go
|   |-- discovery/
|   |   |-- lan.go
|   |   |-- registry.go
|   |   |-- cache.go
|   |   `-- resolver.go
|   |-- scheduler/
|   |   |-- health.go
|   |   |-- dispatch.go
|   |   `-- reschedule.go
|   |-- worker/
|   |   |-- receiver.go
|   |   |-- nix.go
|   |   |-- sandbox.go
|   |   |-- executor.go
|   |   `-- result.go
|   |-- monitor/
|   |   |-- jobs.go
```

```
|   |   `-- ping.go
|   `-- nats/
|       `-- client.go
|
|-- registry/
|   |-- main.go
|   `-- store.go
|
|-- examples/
|   |-- hello.py
|   `-- two_node_local.py
|
|-- go.mod
|-- go.sum
|-- Makefile
`-- README.md
```

5.1.2 Discovery Subsystem

The discovery subsystem is implemented across four files in `daemon/discovery/`. `lan.go` implements mDNS-based LAN multicast discovery using Go's mDNS library. `registry.go` implements the registry client, handling both peer self-registration and peer list retrieval. `cache.go` manages serialisation and deserialisation of the last-known peer list to and from disk. `resolver.go` launches all three discovery sources concurrently and merges their results into a single deduplicated candidate list using goroutines and channels.

5.1.3 Scheduler and Health Subsystem

The scheduler subsystem is implemented in `daemon/scheduler/`. `health.go` queries each candidate worker for CPU utilisation, RAM usage, and active job count, and computes the `HealthScore` using the penalty model described in Section 4.2.1. `dispatch.go` selects the best worker by score, packages the task payload (script, working directory, `.nar`), and pushes it to the worker over NATS. `reschedule.go` handles worker failure detection: it monitors the NATS connection and open task timeouts, and re-dispatches to the next-best worker if a failure is detected.

5.1.4 Worker Subsystem

The worker subsystem is implemented in `daemon/worker/`. `receiver.go` is the NATS listener that accepts incoming task payloads. `nix.go` handles `.nar` import and Nix store path resolution. `sandbox.go` constructs and tears down the bubblewrap namespace,

configuring bind mounts for the working directory and Nix environment. `executor.go` runs the Python script inside the sandbox and captures its output. `result.go` zips the job directory, signals the sender, and serves the download.

5.1.5 Monitor Subsystem

`daemon/monitor/jobs.go` maintains the task history store and implements the tabular output for `pipedpeer jobs`. `daemon/monitor/ping.go` implements the live terminal dashboard for `pipedpeer ping`, refreshing continuously with real-time node status, `HealthScore`, CPU, RAM, and active job counts.

Listing 5.2: `pipedpeer jobs` output

```
$ pipedpeer jobs

  TASK ID         STATUS       NODE                STARTED
     DURATION
  task-0a3f...    complete     worker-lan-02       14:22:01      4.2s
  task-1c9b...    complete     worker-lan-02       14:22:09      3.8s
  task-2e7d...    running      worker-lan-01       14:22:17      --
  task-3b2a...    failed       worker-lan-03       14:21:55      1.1s
```

Listing 5.3: `pipedpeer ping` live dashboard output

```
$ pipedpeer ping

  NODE              STATUS    HEALTH    CPU     RAM      JOBS
  worker-lan-01     online    0.91      12%     34%      1
  worker-lan-02     online    0.74      38%     51%      2
  worker-lan-03     online    0.55      61%     70%      3
  registry-node     online    --        --      --       --
```

5.1.6 Registry Server

The optional registry server is implemented in `registry/`. `store.go` maintains an in-memory peer registration store with automatic expiry of stale entries. `main.go` exposes the HTTP registration and peer list endpoints. The registry holds no task state, no queues, and no results — it is purely a coordination aid for cross-subnet peer discovery.

5.2 Results

5.2.1 Functional Test Results

[Run the following test cases and fill in PASS/FAIL and observed output.]

Table 5.1: Functional Test Results

gray!20 **Test ID**	**Description**	**Result**	**Notes**
TC-01	Smoke test: `hello.py` dispatched to remote worker, output file returned to local folder	[PASS/FAIL]	[Observed output]
rowgray TC-02	Two-node local: cross-dispatch between two daemons on the same machine (`two_node_local.py`)	[PASS/FAIL]	[Observed output]
TC-03	Health-based routing: worker with lower CPU/RAM selected over a busier node	[PASS/FAIL]	[HealthScore values observed]
rowgray TC-04	Local fallback: all remote workers unreachable; task executes locally	[PASS/FAIL]	[Fallback triggered?]
TC-05	Registry failure: registry taken offline mid-session; dispatch continues via mDNS + cache	[PASS/FAIL]	[Discovery degraded gracefully?]
rowgray TC-06	Nix cache reuse: same `.nar` submitted twice; second import completes faster	[PASS/FAIL]	[Import time first vs second run (ms)]
TC-07	Sandbox isolation: script attempts to write outside working directory; write blocked	[PASS/FAIL]	[Observed error / file not created outside sandbox?]
rowgray TC-08	Worker crash recovery: worker goes offline mid-task; sender reschedules on next-best worker	[PASS/FAIL]	[Re-dispatch observed?]
TC-09	Zero-config join: new node joins via mDNS only (no registry); receives task within `[X]` seconds	[PASS/FAIL]	[Time to first task (s)]

5.2.2 Performance Benchmarks

[Run the following benchmarks and fill in the measured values.]

Table 5.2: Performance Benchmark Results

gray!20 **Benchmark**	**Configuration**	**Measured Value**	**Notes**
End-to-end task latency (`hello.py`)	2-node LAN	[ ] ms	Includes dispatch, exec, result retrieval
rowgray `.nar` import — cache miss	First submission	[ ] ms	Full import of [X] packages
`.nar` import — cache hit	Second submission	[ ] ms	Zero unpack; store path reuse
rowgray Health query round-trip	LAN, 3 workers	[ ] ms	Time to score all candidates
Peer discovery latency (mDNS)	LAN	[ ] ms	Time from daemon start to first peer
rowgray Worker failure detection + reschedule	2-node LAN	[ ] s	Time from disconnect to re-dispatch
Task throughput (sequential)	1 worker	[ ] tasks/min	Baseline
rowgray Task throughput (concurrent)	[N] workers	[ ] tasks/min	Throughput scaling

5.2.3 HealthScore Behaviour Validation

The HealthScore system was validated by artificially loading a worker node (running a CPU-intensive background process) and observing whether the sender correctly routes tasks to a less-loaded peer.

[INSERT OBSERVED VALUES: (1) HealthScore of loaded worker at [X]% CPU; (2) HealthScore of idle worker; (3) Confirm that the idle worker was selected for dispatch.]

5.3 Comparative Analysis

The following table compares PipedPeer against existing distributed task execution and compute frameworks across the dimensions most relevant to the problem statement.

Table 5.3: Comparative Analysis of PipedPeer vs Existing Frameworks

gray!20 **Feature**	**PipedPeer**	**Apache Spark**	**Ray**	**Dask**	**BOINC**
Zero-config setup	Yes	No	No	No	No
rowgray No central master node	Yes	No	No	No	No
Unprivileged sandboxing	Yes (bwrap)	No	No	No	No
rowgray Reproducible environments	Yes (Nix)	No	No	No	No
Single binary install	Yes	No	No	No	No
rowgray Push-based scheduling	Yes	Pull	Pull	Pull	Pull
LAN-first discovery	Yes (mDNS)	No	No	No	No
rowgray Automatic local fallback	Yes	No	No	No	No

Apache Spark is the most widely deployed distributed computing framework in industry, but requires a pre-configured, trusted cluster with a dedicated master node. It offers no mechanism for zero-configuration LAN deployment, no sandboxing, and no reproducible environment management. Ray improves on Spark for Python workloads but still requires a head node and a pre-existing cluster.

Dask provides distributed Python computing with a similar API to NumPy/Pandas, but again requires a scheduler process and explicit cluster setup. Like Spark and Ray, it does not address LAN-first discovery, sandboxed execution, or environment portability.

BOINC is the closest system in spirit — it enables volunteer compute donation. However, it is a platform for specific scientific projects, not a general-purpose programmable task executor. Each BOINC project requires its own server infrastructure and custom client software. It offers no sandboxing, no environment management, and no zero-config deployment for general scripts.

PipedPeer fills the gap between these extremes: it provides the simplicity of `ssh script.py` execution without manual configuration, with the reliability guarantees of a production scheduler (health-aware routing, automatic fallback, crash recovery) and the security of sandboxed, reproducible execution environments.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This project has designed and implemented **PipedPeer** — a distributed task execution daemon that enables any machine on a local network to transparently offload Python script execution to the best available remote node, with zero configuration beyond running a single binary.

The key technical contributions of this work are:

1. **Push-Based Health-Aware Dispatch:** A real-time worker selection mechanism that computes a `HealthScore` for every discovered worker based on CPU utilisation, RAM pressure, and active job count, and dispatches tasks to the highest-scoring node over NATS.
2. **Concurrent Three-Source Peer Discovery:** A robust discovery subsystem that merges LAN mDNS multicast, an optional central registry, and a disk-persisted peer cache simultaneously, ensuring that task dispatch continues even when the registry is unreachable.
3. **Nix Content-Addressed Environment Packaging:** A dependency management approach using `.nar` archives that ships the complete execution environment with the task, enabling exact reproducibility and zero-cost cache reuse on workers that have seen the same package set before.
4. **bubblewrap Unprivileged Sandboxing:** Per-task filesystem, process, and network isolation using Linux user namespaces, requiring no root privileges or container daemon on worker machines.
5. **Hard Local Fallback Guarantee:** A fault tolerance mechanism that ensures every submitted task always produces a result by falling back to local execution when no healthy remote worker can be found.

The system achieves all stated objectives for LAN deployment. *[INSERT SUMMARY OF KEY BENCHMARK RESULTS — e.g., end-to-end task latency, .nar cache reuse speedup, peer discovery time.]*

6.2 Limitations of the Current System

The following limitations exist in the current implementation:

- **LAN-only scope:** mDNS discovery does not function across public internet subnets. Nodes on different networks can only discover each other via the registry or manual peer configuration.
- **Python-only runtime:** The current implementation supports Python scripts only. Support for other runtimes requires extending the environment packaging and invocation logic.
- **Sender availability:** If the sender node goes offline during task execution, the worker cannot deliver its result. Sender fault tolerance is deferred to future work.
- **Linux-only workers:** bubblewrap and Nix are Linux-specific. Worker nodes must run Linux; the sender may run any platform.
- **No incentive mechanism:** Workers contribute compute voluntarily. There is no reward for providing resources, limiting deployment to cooperative or trusted environments.
- **Sequential task model:** The current system dispatches one task per `pipedpeer run` invocation. Parallel multi-task job splitting is not yet implemented.

6.3 Future Work

The following enhancements are planned as direct continuations of this work:

- **Internet-scale Kademlia DHT peer discovery:** Replacing or augmenting mDNS with a DHT will allow nodes anywhere on the internet to discover each other without a shared LAN, removing the current LAN-only constraint.
- **NAT traversal:** Adding hole-punching or relay support will allow nodes behind home routers to establish direct NATS connections without port forwarding.
- **Multi-language runtime support:** Extending the environment packaging and invocation logic to support additional runtimes (e.g., Node.js, R, Julia) by parameterising the execution entry point.
- **Multi-task job splitting:** Allowing a single `pipedpeer run` invocation to split a workload across multiple workers concurrently, with result aggregation on the sender side.
- **Sender fault tolerance via DHT-stored results:** Workers could push completed results to a distributed store keyed by task ID, allowing a reconnecting sender to retrieve

results without re-execution.

- **Token-based incentive system:** A per-task micropayment mechanism to reward workers for verified computation, enabling participation from untrusted or external contributors.
- **macOS and Windows worker support:** Replacing bubblewrap with a cross-platform sandboxing abstraction (e.g., WASM or platform-specific sandbox APIs) would allow worker daemons to run on non-Linux hosts.

Chapter 7

Research Paper

PipedPeer: A Push-Based Distributed Task Execution Daemon with Nix Environment Portability and bubblewrap Sandboxing

Yateen Vaviya, Vinayak Yadav, Sahil Gupta — Sardar Patel Institute of Technology

---

Abstract

We present **PipedPeer**, a distributed task execution daemon that enables a local network of heterogeneous machines to transparently share compute resources for Python script execution. PipedPeer requires no cluster pre-configuration, no root privileges, and no software installation beyond a single Go binary. The system uses a push-based dispatch model in which the sender evaluates real-time worker health (CPU, RAM, active jobs) and directly delivers the task to the best available node over NATS. Execution environment reproducibility is achieved through Nix content-addressed archives (`.nar`), enabling zero-cost cache reuse on repeat invocations. Process isolation is enforced per-task using bubblewrap unprivileged Linux namespaces. A three-source peer discovery subsystem (LAN mDNS, optional central registry, disk-persisted cache) ensures dispatch continues even under partial network failure. A hard local-fallback guarantee ensures every submitted task always produces a result. We evaluate the system on a LAN of *[N]* nodes and report end-to-end task latency of *[X]* ms, `.nar` cache-hit setup time of *[Y]* ms, and peer discovery latency of *[Z]* ms.

---

1. Introduction

The gap between a local `python script.py` and a distributed cluster job is enormous in operational complexity but often small in the actual computation involved. Existing distributed frameworks — Apache Spark [1], Ray [2], Dask [4] — require pre-configured clusters, trusted networks, and dedicated scheduler processes. Remote execution via `ssh`

is manual, provides no environment management, and offers no automatic worker selection or sandboxing.

PipedPeer eliminates this gap. Its design philosophy is that distributed execution should be as simple as local execution: the user runs `pipedpeer run script.py` and output files appear in their working directory, regardless of where the computation actually occurred.

The core contributions of this paper are: (1) a push-based, health-aware dispatch model that routes tasks to the best available worker with no pre-configuration; (2) Nix `.nar`-based environment portability with content-addressed cache reuse; (3) per-task bubblewrap sandboxing requiring no root or container runtime; (4) a three-source concurrent peer discovery subsystem with graceful degradation; and (5) a hard local-fallback guarantee ensuring task completion under all network conditions.

2. System Design

PipedPeer runs as a background daemon on every node. The CLI is a thin client with no peer-to-peer logic. On task submission, the daemon's discovery subsystem concurrently queries LAN mDNS, an optional registry [**?**], and a disk-persisted peer cache, merging results into a candidate list. Each candidate is queried for real-time health metrics; a `HealthScore` is computed and the task is pushed to the highest-scoring worker over NATS [6]. If no healthy worker is found, the task runs locally. The worker imports the `.nar` environment (reusing cached Nix [5] store paths), creates a bubblewrap [7] sandbox, executes the script, zips the output directory, and signals the sender. The sender extracts the result into the user's local folder.

3. The HealthScore Model

The HealthScore assigned to each candidate worker w is:

$$H(w) = \max(0,\ 1.0 - \alpha_{\text{cpu}} \cdot u_{\text{cpu}}(w) - \alpha_{\text{ram}} \cdot u_{\text{ram}}(w) - \alpha_{\text{jobs}} \cdot n_{\text{jobs}}(w)) \tag{7.1}$$

where u_{cpu} and u_{ram} are the current CPU and RAM utilisation fractions, n_{jobs} is the number of active jobs, and α_{cpu}, α_{ram}, α_{jobs} are configurable penalty weights. Workers that do not respond to the health query within a timeout are assigned $H = 0$. The sender dispatches to $\arg\max_w H(w)$.

4. Environment Portability via Nix .nar

Nix represents packages as content-addressed store paths. A `.nar` archive serialises one or more store paths. On import, the worker checks each store path hash against its local store; paths already present are reused at $O(1)$ cost. This means the first execution of

a given environment pays the full import cost; all subsequent executions with the same dependency set pay nothing for setup. This is analogous to a build cache in continuous integration systems, but applied at the package granularity of Nix.

5. Sandboxing via bubblewrap

bubblewrap creates an unprivileged Linux user namespace for each task. Only two filesystem paths are visible inside the sandbox: the job working directory (read-write) and the Nix environment (read-only). No host system paths are exposed. Process isolation ensures the script's process tree is the only one visible inside the namespace. Network access inherits the host by default but can be restricted per-task. The entire sandbox setup and teardown occurs within the `pipedpeer` binary with no external daemon required.

6. Evaluation

[INSERT EVALUATION SECTION: Include tables/figures for: (1) end-to-end task latency on LAN at varying worker counts; (2) .nar import time — cache miss vs cache hit; (3) peer discovery latency for mDNS vs registry vs cache; (4) HealthScore routing accuracy — tasks routed to lowest-load worker % of time. Describe hardware specs of test nodes, network conditions, number of trials, and error bars.]

7. Related Work

Apache Spark [1] and Ray [2] are the dominant distributed computing frameworks but require pre-configured, trusted clusters. Dask [4] extends Python to distributed execution but similarly requires an explicit scheduler. BOINC [3] pioneered volunteer computing but is a project-specific platform requiring dedicated server infrastructure. Nix [5] introduced purely functional package management with content-addressed builds; PipedPeer leverages its `.nar` serialisation for portable execution environments. NATS [6] provides the lightweight pub-sub transport; its low-latency, broker-optional design suits PipedPeer's direct node-to-node dispatch model. bubblewrap [7] provides the sandboxing primitive enabling unprivileged namespace isolation without a container runtime.

8. Conclusion

PipedPeer demonstrates that zero-configuration, sandboxed, reproducible distributed task execution is achievable with a single-binary deployment model on a LAN. The push-based health-aware dispatch model eliminates the need for a central scheduler. Nix `.nar` archives make execution environments portable and cache-efficient. bubblewrap sandboxing provides security without operational overhead. The hard local-fallback guarantee means users never lose a task. Future work will extend the system to internet-scale via DHT

discovery and NAT traversal, add multi-language runtime support, and implement a token-based incentive model for untrusted participant networks.

Chapter 8

Plagiarism Report

The plagiarism report for this project report was generated using *[INSERT TOOL NAME — e.g., Turnitin / iThenticate / Urkund / DRILLBIT]* on *[INSERT DATE]*.

Table 8.1: Plagiarism Check Summary

gray!20 **Parameter**	**Value**	**Notes**
Tool Used	[Tool Name]	
rowgray Date of Check	[DD/MM/YYYY]	
Similarity Index	[ ]%	Overall similarity percentage
rowgray Internet Sources	[ ]%	Matched to web content
Publications	[ ]%	Matched to journal/conference papers
rowgray Student Papers	[ ]%	Matched to other student submissions

[ATTACH OR INSERT: Screenshot / exported PDF of the plagiarism report from your institution's approved tool. Most institutions require the similarity index to be below 20% (or as specified by your department).]

Bibliography

[1] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, "Spark: Cluster computing with working sets," *Proc. 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*, 2010.

[2] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan, and I. Stoica, "Ray: A distributed framework for emerging AI applications," *Proc. 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018, pp. 561–577.

[3] D. P. Anderson, "BOINC: A system for public-resource computing and storage," *Proc. 5th IEEE/ACM International Workshop on Grid Computing*, 2004, pp. 4–10.

[4] M. Rocklin, "Dask: Parallel computation with blocked algorithms and task scheduling," *Proc. 14th Python in Science Conference (SciPy)*, 2015, pp. 130–136.

[5] E. Dolstra, M. de Jonge, and E. Visser, "Nix: A safe and policy-free system for software deployment," *Proc. 18th Large Installation System Administration Conference (LISA)*, 2004, pp. 79–92.

[6] Synadia Communications, "NATS: Cloud native messaging," [Online]. Available: https://nats.io. [Accessed: 2024].

[7] Project Atomic, "bubblewrap: Unprivileged sandboxing tool," [Online]. Available: https://github.com/containers/bubblewrap. [Accessed: 2024].

[8] A. B. Kahn, "Topological sorting of large networks," *Communications of the ACM*, vol. 5, no. 11, pp. 558–562, 1962.

[9] P. Maymounkov and D. Mazières, "Kademlia: A peer-to-peer information system based on the XOR metric," *Proc. 1st International Workshop on Peer-to-Peer Systems (IPTPS)*, 2002, pp. 53–65.

[10] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," *Proc. 6th Symposium on Operating Systems Design and Implementation (OSDI)*, 2004, pp. 137–150.

[11] [Author(s)], "[Paper Title — Placeholder]," *[Journal / Conference Name]*, [Year].

[12] [Author(s)], "[Paper Title — Placeholder]," *[Journal / Conference Name]*, [Year].